Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа № 3
«Организация информационного взаимодействия по интерфейсу I2C»

по дисциплине

«Проектирование вычислительных систем»

Вариант № 5: «Кодовый замок»

Выполнили: Ястребов-Амирханов Алекси, Исупов Никита Александрович
Группа: P3432/P3431
Преподаватель: Ключев Владислав Аркадьевич

Санкт-Петербург,

2026

Оглавление

1. Цель работы **3**
2. Задание и вариант **3**
3. Аппаратная часть и используемые интерфейсы **3**
3.1. Учебный стенд SDK-1.1M и шина I2C 3
3.2. Графический OLED-дисплей 128×64 (контроллер SSD1306) 4
3.3. Энергонезависимая память EEPROM (микросхема MCP79411) 4
3.4. Интерфейс связи с персональным компьютером (USART6) 4
4. Программная реализация прошивки микроконтроллера **5**
4.1. Архитектура и конечный автомат (FSM) кодового замка 5
4.2. Алгоритм работы с энергонезависимой памятью EEPROM 6
4.3. Вывод графической информации и маски пароля на OLED 6
4.4. Текстовый протокол информационного обмена с ПК 7
5. Программная реализация графического интерфейса пользователя (GUI) **7**
5.1. Назначение и возможности приложения client_gui.py 7
5.2. Многопоточный прием данных и файл журнала (логирование) 8
6. Результаты тестирования и верификации системы **8**
7. Заключение **9**

1. Цель работы

Изучить принципы аппаратной и программной организации последовательного интерфейса I2C (Inter-Integrated Circuit) в микроконтроллерах семейства STM32F4, освоить работу с физическими периферийными контроллерами шины, осуществить подключение и взаимодействие с графическим OLED-дисплеем и микросхемой энергонезависимой памяти EEPROM, а также разработать клиентское программное обеспечение для персонального компьютера с графическим интерфейсом пользователя (GUI) для управления комплексом и регистрации событий в журнале.

2. Задание и вариант

Вариант № 5: «Кодовый замок». В рамках задания требуется реализовать встраиваемую систему электронного кодового замка со следующими техническими характеристиками и функциональными возможностями:

- Хранение мастер-пароля в энергонезависимой памяти EEPROM с сохранением конфигурации при полном отключении питания стенда;
- Отображение состояния замка (заблокирован, ожидание ввода, доступ разрешен, ошибка) и маски ввода пароля на графическом монохромном OLED-дисплее (128×64 точки);
- Защита от перебора комбинаций: при фиксации трех подряд неверных попыток ввода замок переходит в режим аппаратной блокировки на 5 секунд с индикацией обратного отсчета;
- Возможность смены активного мастер-пароля с последующей записью нового значения в микросхему EEPROM;
- Двусторонний обмен данными с персональным компьютером по интерфейсу UART (USART6) со скоростью 9600 бод;
- Разработка клиентской программы для ПК с графическим интерфейсом (GUI), обеспечивающей удаленный ввод кода, изменение настроек и протоколирование всех транзакций в файл журнала.

3. Аппаратная часть и используемые интерфейсы

3.1. Учебный стенд SDK-1.1M и шина I2C

В лабораторной работе используется лабораторный стенд SDK-1.1M, построенный на базе 32-разрядного микроконтроллера STM32F427VIT6 (архитектура ARM Cortex-M4, тактовая частота до 180 МГц). Связь микроконтроллера со внешними ведомыми устройствами (Slave) реализована посредством аппаратного модуля I2C1.

Шина I2C является синхронной двухпроводной полудуплексной шиной последовательной передачи данных. Для передачи сигналов используются две линии: SCL (Serial Clock — тактовая линия) и SDA (Serial Data — линия данных). Обе физические линии организованы по схеме с открытым стоком (Open-Drain) и подтянуты резисторами 4.7 кОм к шине питания микроконтроллера +3.3 В («монтажное И»). В проекте задействованы выводы PB8 (I2C1_SCL) и PB9 (I2C1_SDA), скорость обмена сконфигурирована в стандартном режиме (Standard Mode, 100 кГц).

Таблица 3.1. Аппаратные узлы стенда SDK-1.1M и параметры адресации

Устройство / Модуль	Интерфейс	Выводы STM32	Адрес на шине / Параметры
Дисплей OLED 128×64	I2C1	PB8 (SCL), PB9 (SDA)	7-bit: 0x3C (HAL: 0x78)
EEPROM (MCP79411)	I2C1	PB8 (SCL), PB9 (SDA)	7-bit: 0x57 (HAL: 0xAE/0xAF)
USB-UART мост (FT2232HL)	USART6	PG14 (TX), PG9 (RX)	9600 8N1, виртуальный COM
RGB-светодиоды индикации	GPIO / TIM	PD12, PD13, PD14, PD15	Выходы индикации замка

3.2. Графический OLED-дисплей 128×64 (контроллер SSD1306)

Для визуального отображения информации используется дисплейный модуль разрешением 128×64 пикселей на базе драйвера SSD1306. Графическая память дисплея (GDDRAM) организована в виде 8 горизонтальных страниц (PAGE0–PAGE7) высотой по 8 бит (1 байт) и шириной 128 столбцов (SEG0–SEG127). Суммарный объем видеопамяти составляет 128 × 64 / 8 = 1024 байта.

При передаче команд по шине I2C первый байт пакета содержит контрольный байт Co/D# (0x00 — поток команд, 0x40 — поток графических данных). Контроллер сконфигурирован в постраничном режиме адресации (Page Addressing Mode), что позволяет последовательно обновлять кадровый буфер целиком.

3.3. Энергонезависимая память EEPROM (микросхема MCP79411)

В качестве энергонезависимой памяти используется микросхема MCP79411 (содержащая блок EEPROM). 7-битный адрес устройства на шине равен 0x57. В библиотеке HAL данный адрес сдвигается на 1 бит влево, формируя 8-битный адрес записи 0xAE (младший бит R/W=0) и адрес чтения 0xAF (младший бит R/W=1).

Критическим физическим параметром микросхемы EEPROM является время внутреннего цикла записи страницы памяти: t_WC ≤ 5 мс. В отличие от чтения (которое происходит мгновенно на частоте шины I2C), при записи микросхема проводит стирание и перепрограммирование ячеек плавающего затвора. Во время этого цикла контроллер памяти не отвечает на обращения по шине и выставляет отрицательное подтверждение (NACK). По этой причине в драйвере eeprom.c после каждой транзакции записи введена обязательная программная задержка HAL_Delay(6).

3.4. Интерфейс связи с персональным компьютером (USART6)

Для сопряжения микроконтроллера со внешней рабочей станцией оператора задействован периферийный модуль USART6 (линии PG14 и PG9), подключенный через аппаратный преобразователь интерфейсов FTDI FT2232HL. Канал B микросхемы FTDI конфигурируется как виртуальный COM-порт операционной системы ПК. Связь осуществляется на скорости 9600 бод, формат кадра: 8 бит данных, 1 стоп-бит, без контроля четности (8N1). Драйвер микроконтроллера осуществляет прием и передачу команд в фоновом режиме без блокировки основного цикла программы.

4. Программная реализация прошивки микроконтроллера

4.1. Архитектура и конечный автомат (FSM) кодового замка

Прошивка микроконтроллера организована по принципу кооперативной диспетчеризации. В бесконечном цикле main() поочередно опрашиваются неблокирующие обработчики модулей: прием символов UART, конечный автомат замка lock_fsm_update(), генератор световых эффектов led_update() и обновление дисплея update_oled_display().

Конечный автомат кодового замка оперирует следующими дискретными состояниями:
1. STATE_INPUT: ожидание ввода комбинации символов, на экране отображаются звездочки или прочерки;
2. STATE_UNLOCKED: доступ разрешен, на 3 секунды зажигается зеленый светодиод и выводится надпись OPENED;
3. STATE_WRONG_FLASH: одиночная ошибка, кратковременная вспышка красного светодиода;
4. STATE_BLOCKED: превышен лимит в 3 ошибки, замок блокируется на 5 секунд с миганием красного диода;
5. STATE_SET_NEW_PASSWORD: ввод новой комбинации для сохранения в энергонезависимую память EEPROM.

```
/* Листинг 4.1. Обработка команд протокола ПК и посимвольного ввода (lock_fsm.c) */
static void handle_protocol_command(const char *cmd)
{
    if (strcmp(cmd, "PING") == 0) {
        uart_driver_write_string("PONG\r\n");
        return;
    }
    if (strncmp(cmd, "AUTH:", 5) == 0) {
        const char *pass = cmd + 5;
        if (s_state == STATE_BLOCKED) {
            uart_driver_write_string("STATUS:BLOCKED\r\n");
            return;
        }
        if (strcasecmp(pass, s_master_password) == 0) {
            s_state = STATE_UNLOCKED;
            s_state_start_tick = HAL_GetTick();
            s_wrong_count = 0;
            led_stop_effects();
            led_set(LED_GREEN, LED_STATE_ON);
            s_screen_dirty = true;
            uart_driver_write_string("STATUS:OK\r\n");
            uart_driver_write_string("DISPLAY:OPENED\r\n");
        } else {
            s_wrong_count++;
            if (s_wrong_count >= MAX_WRONG_ATTEMPTS) {
                s_state = STATE_BLOCKED;
                s_state_start_tick = HAL_GetTick();
                led_start_blink(LED_RED, 200, BLOCK_DURATION_MS);
                uart_driver_write_string("STATUS:BLOCKED\r\n");
            } else {
                s_state = STATE_WRONG_FLASH;
                s_state_start_tick = HAL_GetTick();
                led_trigger_flash(LED_RED, 500);
                uart_driver_write_string("STATUS:ERR\r\n");
            }
            s_screen_dirty = true;
            uart_driver_write_string("DISPLAY:ERROR\r\n");
```

```
        }
        return;
    }
    if (strncmp(cmd, "SET:", 4) == 0) {
        const char *new_p = cmd + 4;
        strncpy(s_master_password, new_p, MAX_PASSWORD_LEN);
        s_master_password[MAX_PASSWORD_LEN] = '\0';
        eeprom_save_password(s_master_password);
        uart_driver_write_string("STATUS:SAVED\r\n");
        s_state = STATE_INPUT;
        s_screen_dirty = true;
        return;
    }
}
```

4.2. Алгоритм работы с энергонезависимой памятью EEPROM

Драйвер энергонезависимой памяти eeprom.c реализует чтение и запись строк переменной длины. При инициализации микроконтроллера функция eeprom_load_password() выполняет считывание байтов с адреса 0x00. Если считанные данные содержат валидные печатные ASCII-символы (код от 0x20 до 0x7E), пароль загружается в оперативную память как активный мастер-пароль. Если память не инициализирована (содержит 0xFF после очистки или мусор), драйвер автоматически прошивает значение по умолчанию «1234».

```
/* Листинг 4.2. Чтение с проверкой сигнатуры и запись в EEPROM по I2C (eeprom.c) */
bool eeprom_write(uint8_t mem_addr, const uint8_t *data, uint16_t len)
{
    if (s_hi2c == NULL || data == NULL || len == 0) return false;
    for (uint16_t i = 0; i < len; i++) {
        HAL_StatusTypeDef status = HAL_I2C_Mem_Write(
            s_hi2c, EEPROM_I2C_ADDR, mem_addr + i,
            I2C_MEMADD_SIZE_8BIT, (uint8_t *)&data[i], 1, 50
        );
        if (status != HAL_OK) return false;
        HAL_Delay(6); /* Обязательное ожидание внутреннего цикла t_WC */
    }
    return true;
}

void eeprom_load_password(char *out_pass, const char *default_pass)
{
    uint8_t buf[EEPROM_PASS_MAX_LEN + 1] = {0};
    bool ok = eeprom_read(EEPROM_PASS_ADDR, buf, EEPROM_PASS_MAX_LEN);
    bool valid = ok && (buf[0] >= 0x20 && buf[0] <= 0x7E);
    if (valid) {
        strncpy(out_pass, (char *)buf, EEPROM_PASS_MAX_LEN);
    } else {
        strncpy(out_pass, default_pass, EEPROM_PASS_MAX_LEN);
        eeprom_save_password(out_pass);
    }
}
```

4.3. Вывод графической информации и маски пароля на OLED

Драйвер oled.c инкапсулирует программный буфер экрана s_oled_buffer размером 1024 байта в оперативной памяти микроконтроллера. Любые графические примитивы (рамки, прямоугольники, символы шрифтов 7×10 и 11×18) сначала рендерятся в ОЗУ, после чего

процедура oled_UpdateScreen() синхронизирует измененные страницы с физическим контроллером.

```
/* Листинг 4.3. Постраничная передача кадрового буфера на дисплей SSD1306 (oled.c) */
void oled_UpdateScreen(void)
{
    for (uint8_t i = 0; i < 8; i++) {
        oled_WriteCommand(0xB0 + i); /* Установка номера страницы (PAGE0..PAGE7) */
        oled_WriteCommand(0x00);      /* Младший ниббл начального столбца = 0 */
        oled_WriteCommand(0x10);      /* Старший ниббл начального столбца = 0 */
        HAL_I2C_Mem_Write(
            &hi2c1, OLED_I2C_ADDR, 0x40,
            I2C_MEMADD_SIZE_8BIT, &s_oled_buffer[SSD1306_WIDTH * i],
            SSD1306_WIDTH, 100
        );
    }
}
```

4.4. Текстовый протокол информационного обмена с ПК

Информационное взаимодействие между ПК и стендом SDK-1.1M организовано посредством текстового протокола поверх последовательного интерфейса UART. Все пакеты представляют собой ASCII-строки, оканчивающиеся символами перевода строки \r\n.

Таблица 4.1. Спецификация команд протокола обмена по интерфейсу UART

Пакет / Команда	Направление	Назначение и реакция системы
PING	ПК -> МК	Проверка активности соединения. Ответ МК: PONG\r\n
AUTH:<pass>	ПК -> МК	Передача пароля. Ответ: STATUS:OK или STATUS:ERR / BLOCKED
SET:<new_pass>	ПК -> МК	Запись нового пароля в EEPROM. Ответ: STATUS:SAVED
GET_STATUS	ПК -> МК	Запрос статуса. Ответ: STATUS:LOCKED / UNLOCKED / BLOCKED
DISPLAY:<text>	МК -> ПК	Телеметрия виртуального OLED: текущая маска или надпись

5. Программная реализация графического интерфейса пользователя (GUI)

5.1. Назначение и возможности приложения client_gui.py

Клиентское приложение для операционной системы Windows разработано на языке Python 3 с использованием стандартной графической библиотеки tkinter и библиотеки прямого асинхронного взаимодействия с последовательными портами pyserial. Программа оформлена в виде специализированной панели управления («Dark Slate Hardware Dashboard»).

Функциональные возможности клиентского приложения включают в себя:

- Автоматическое сканирование доступных COM-портов в диспетчере устройств и подключение на скорости 9600 8N1;
- Крупная цветографическая индикация текущего состояния замка: ЗАКРЫТО (синий), ОТКРЫТО (зеленый), БЛОКИРОВКА (красный);
- Виртуальный дисплей стенда: в режиме реального времени отображает маску ввода пароля и телеметрию экрана SSD1306;
- Интерактивная цифровая клавиатура (Numpad) для ввода символов мышью или с физической клавиатуры ПК;
- Панель прямого управления энергонезависимой памятью EEPROM: поле ввода нового мастер-пароля с кнопкой записи;
- Встроенное терминальное окно с цветовой маркировкой входящего ([RX]) и исходящего ([TX]) трафика и автоскроллингом.

5.2. Многопоточный прием данных и файл журнала (логирование)

Для обеспечения абсолютной отзывчивости графического интерфейса и предотвращения блокировки главного потока (GUI Thread) опрос последовательного порта вынесен в отдельный независимый поток-демон (Daemon Thread). Поток непрерывно вычитывает байты из порта по символам перевода строки, производит разбор протокольных префиксов (STATUS:, DISPLAY:) и через потокобезопасную очередь root.after() передает данные в интерфейс.

Параллельно все события, включая системные отметки времени с точностью до миллисекунд, направление передачи ([TX] / [RX]) и текст сообщений, автоматически протоколируются в сессионный файл журнала (например, uart_session_20261102_150527.log), сохраняемый в рабочей директории.

```
# Листинг 5.1. Асинхронный поток чтения COM-порта и разбор протокола (client_gui.py)
def _serial_reader_loop(self):
    while self.running and self.ser and self.ser.is_open:
        try:
            raw_line = self.ser.readline()
            if not raw_line: continue
            line = raw_line.decode('utf-8', errors='ignore').strip()
            if not line: continue
            self._log_transaction('RX', line)
            if line.startswith('STATUS:'):
                status_val = line.split(':', 1)[1]
                self.root.after(0, self._apply_status, status_val)
            elif line.startswith('DISPLAY:'):
                disp_val = line.split(':', 1)[1]
                self.root.after(0, self._apply_display, disp_val)
        except Exception as e:
            break
```

6. Результаты тестирования и верификации системы

В ходе лабораторных испытаний программно-аппаратного комплекса были проведены следующие экспериментальные тесты:

- Тестирование шины I2C сканером адресов: подтверждено устойчивое обнаружение контроллера дисплея SSD1306 (адрес 0x3C) и микросхемы EEPROM MCP79411 (адрес 0x57) без коллизий на шине;
- Тестирование энергонезависимого сохранения мастер-пароля: через GUI-клиент был задан новый пароль «5555», сохраненный в EEPROM. После полного отключения стенда от питания USB и повторного включения замок успешно открылся введенным паролем «5555», подтвердив корректность алгоритма энергонезависимого хранения;
- Тестирование защитной блокировки от перебора: произведен трехкратный ввод заведомо некорректной комбинации «0000». Замок перешел в режим STATUS:BLOCKED на 5 секунд, включилась световая индикация ошибки, ввод с клавиатуры блокировался до истечения защитного интервала;
- Тестирование информационного обмена с GUI: проверена передача команд PING/PONG, AUTH, SET, а также непрерывная запись транзакций в сессионный файл журнала uart_session_*.log.

7. Заключение

В ходе выполнения лабораторной работы №3 были всесторонне исследованы принципы функционирования последовательного синхронного интерфейса I2C в микроконтроллерах семейства STM32F4.

Освоены механизмы арбитража шины, условия формирования сигналов START, STOP, байта адресации и подтверждения ACK/NACK. С использованием аппаратного контроллера I2C1 микроконтроллера STM32F427VIT6 успешно реализована интеграция с графическим OLED-дисплеем SSD1306 (разрешение 128×64) и микросхемой энергонезависимой памяти EEPROM MCP79411. Реализован корректный алгоритм постраничной записи данных в EEPROM с выдержкой технологической временной задержки t_WC = 6 мс.

Разработан отказоустойчивый конечный автомат кодового замка со встроенной защитой от подбора пароля (блокировка на 5 секунд при трех ошибках ввода подряд). Создано клиентское программное обеспечение client_gui.py с графическим интерфейсом пользователя на Python/tkinter, обеспечивающее двусторонний информационный обмен по интерфейсу UART, удаленное управление, синхронизацию отображения виртуального OLED-экрана и сохранение истории всех событий в файл сессионного журнала.